Introduction to Parallel Computing 1

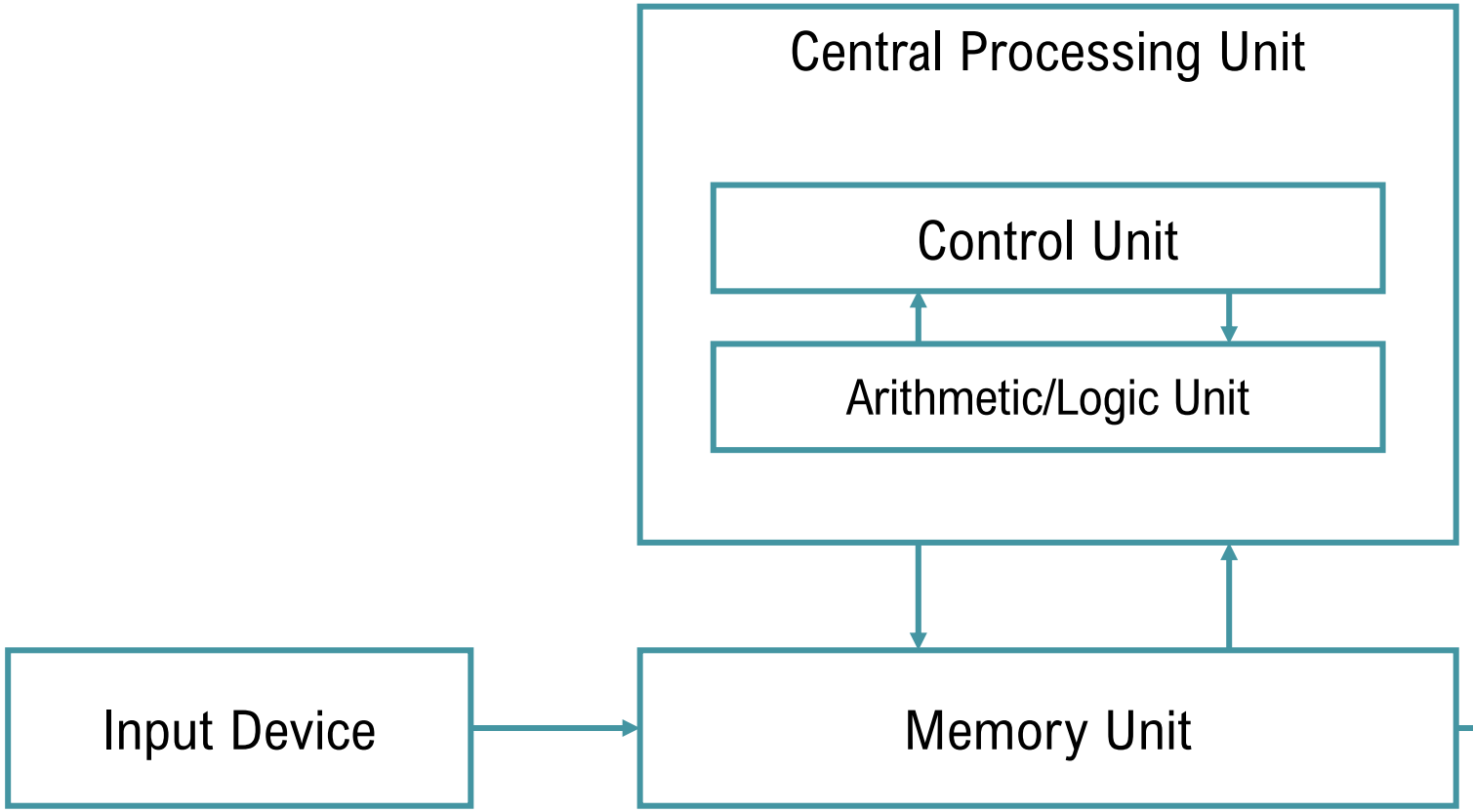
Reference: “Introduction to Parallel Computing”, Chapters 1 and 2

Agenda

- Classification of Parallel Architectures
  - Flynn's classification
  - Communication model-based classification
    - Shared memory
    - Message passing
  - Execution model-based classification
  - Other classifications

Classification of Parallel Architectures

Review: The Von Neumann Architecture

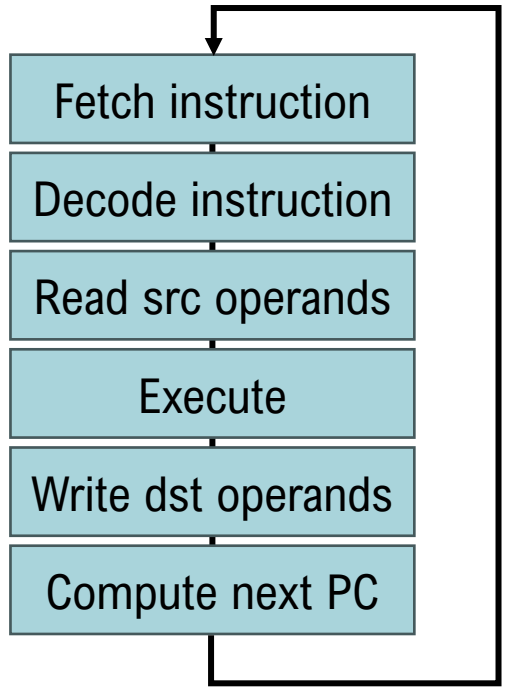


Pre-Von Neumann Architecture
- Input device and memory for data existed
- Programs were hard-wired

Program instructions stored in a memory array
The same memory unit for program and data

Review: Instructions

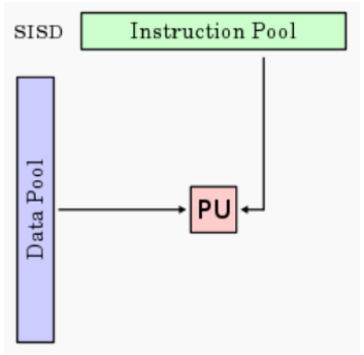
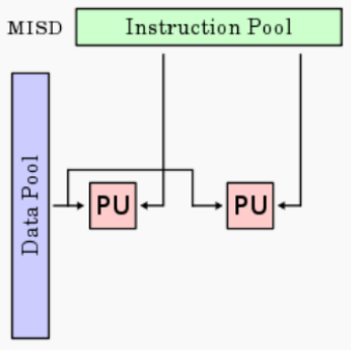
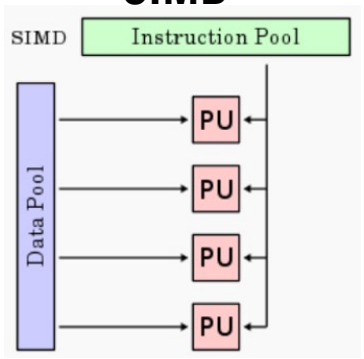
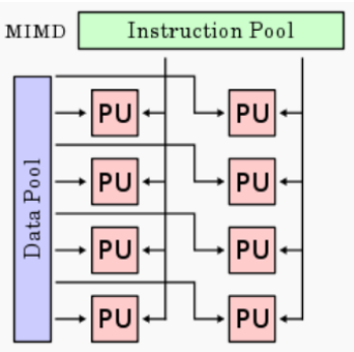
- An instruction is the most basic unit of computer processing
  - Instructions are words in the language of a computer
  - Instruction Set Architecture (ISA) is the vocabulary



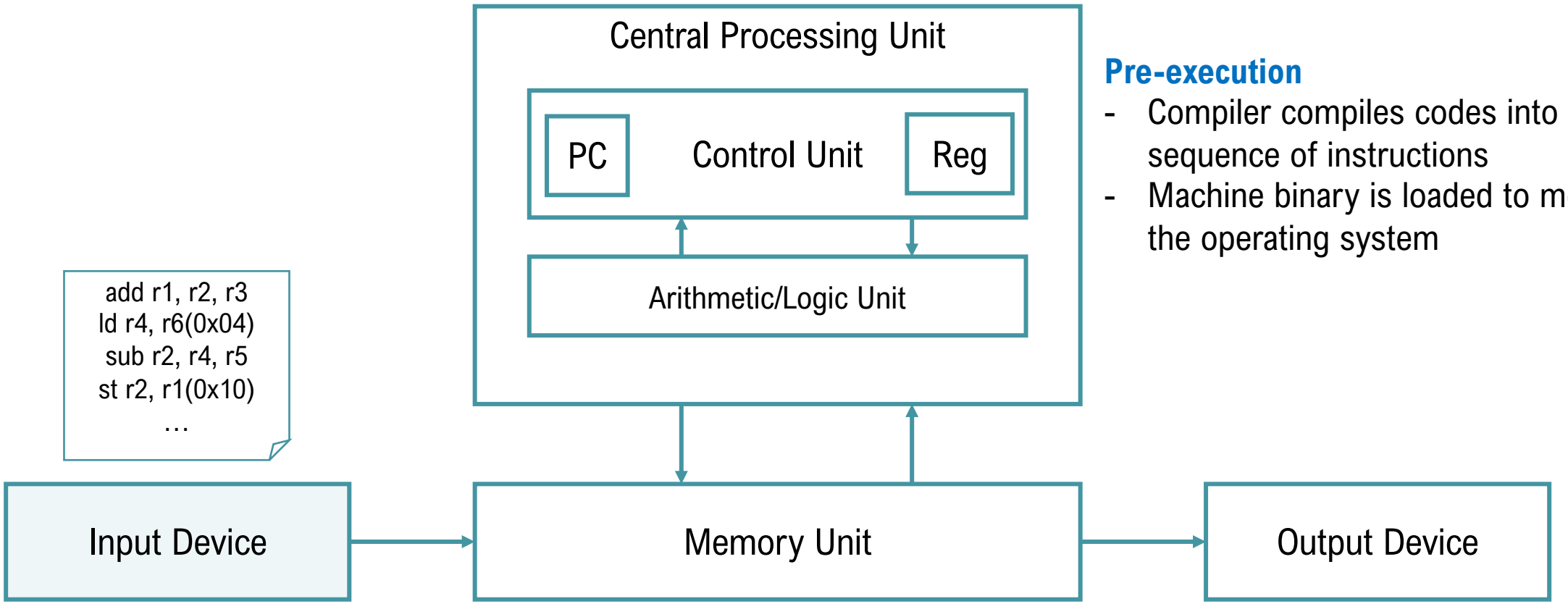
- The language of the computer can be written as
  - Machine language: Computer-readable representation (that is, 0's and 1's)
  - Assembly language: Human-readable representation

- In a von Neumann machine, instructions are executed sequentially
  - CPU logically implements this loop
  - By default, the next PC (program count) is current PC + size of current instruction unless the instruction says otherwise

Flynn's Taxonomy

	Single instruction stream	Multiple instruction streams
Single data stream	**SISD**  e.g., single-core CPU	**MISD**  e.g., fault-tolerant, safety-critical systems
Multiple data streams	**SIMD**  e.g., GPU core, vector processors	**MIMD**  e.g., multi-core CPUs, Multiple CPUs

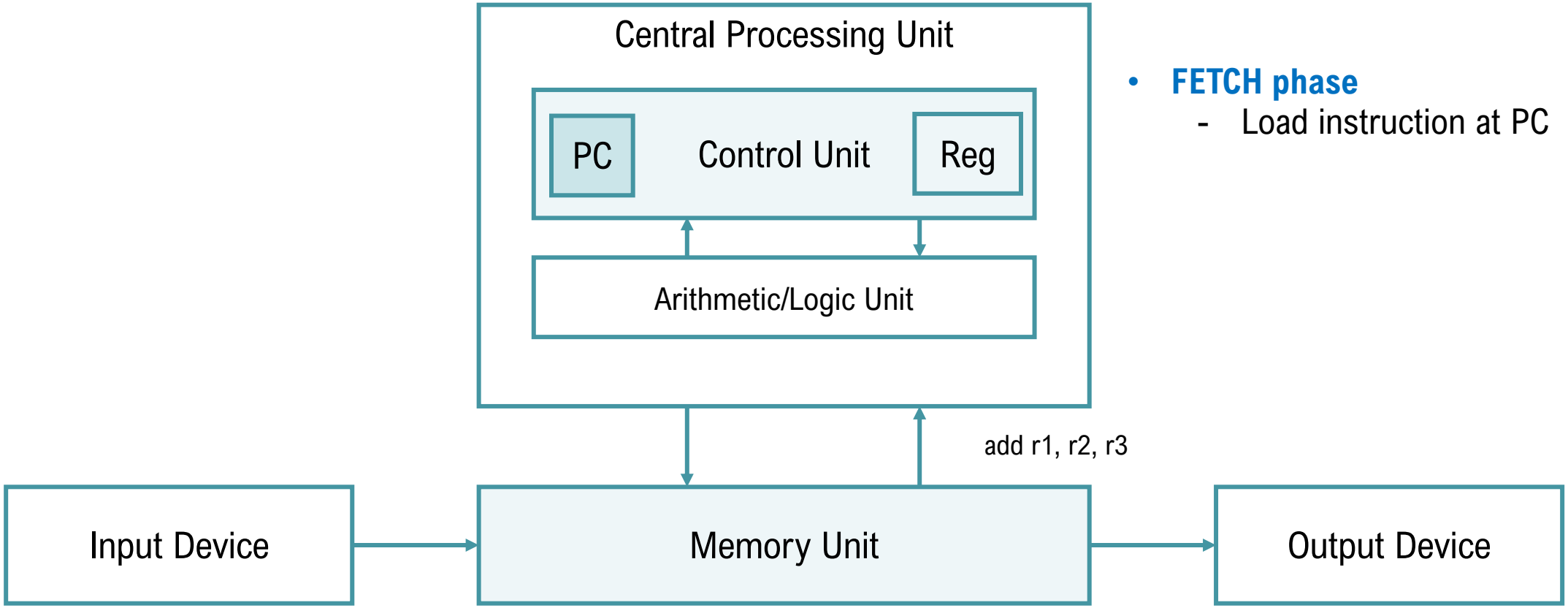
Single-Instruction, Single-Data (SISD)



Pre-execution

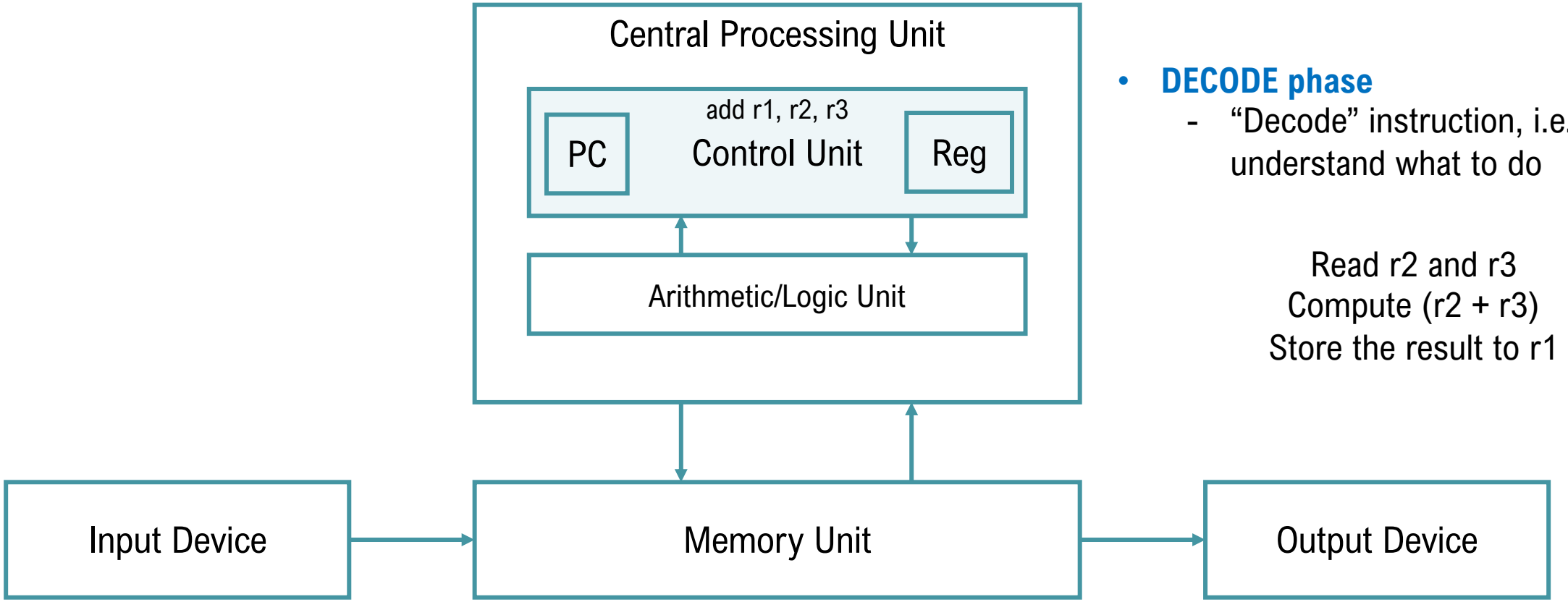
- Compiler compiles codes into a sequence of instructions
- Machine binary is loaded to memory by the operating system

Single-Instruction, Single-Data (SISD)



- **FETCH phase**
  - Load instruction at PC

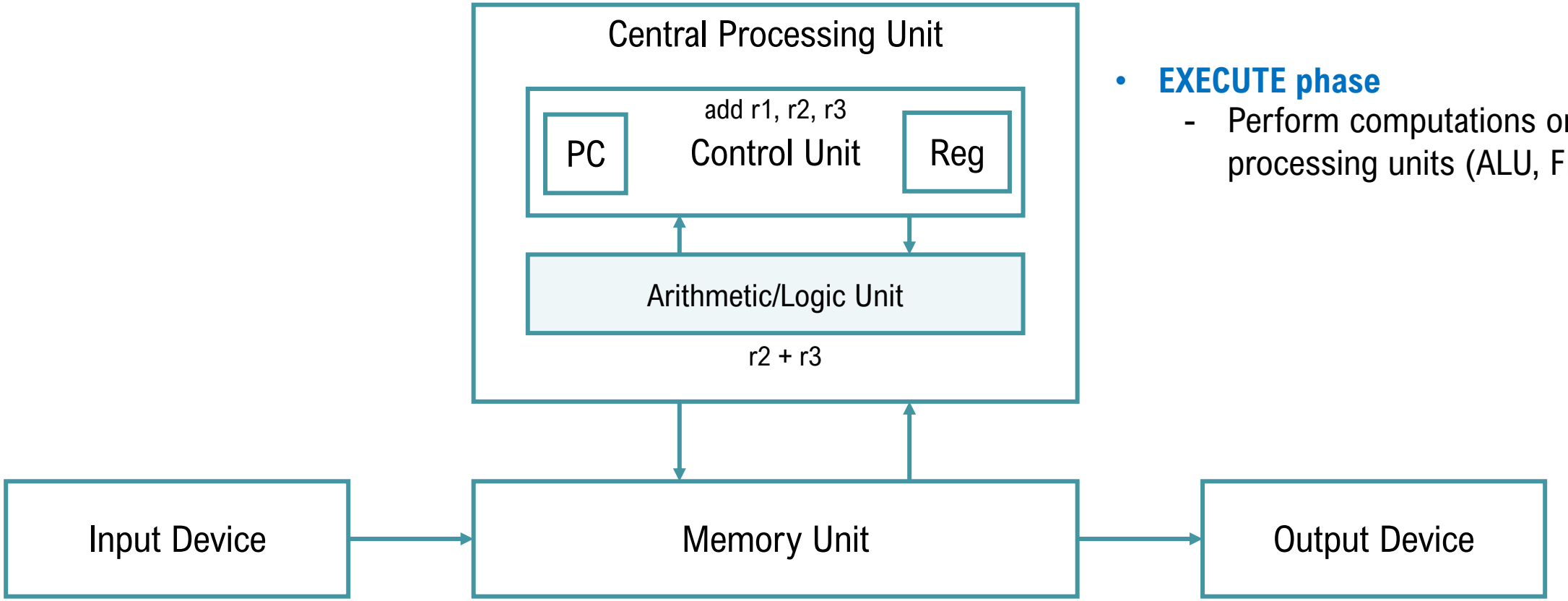
Single-Instruction, Single-Data (SISD)



- **DECODE phase**
  - "Decode" instruction, i.e., understand what to do

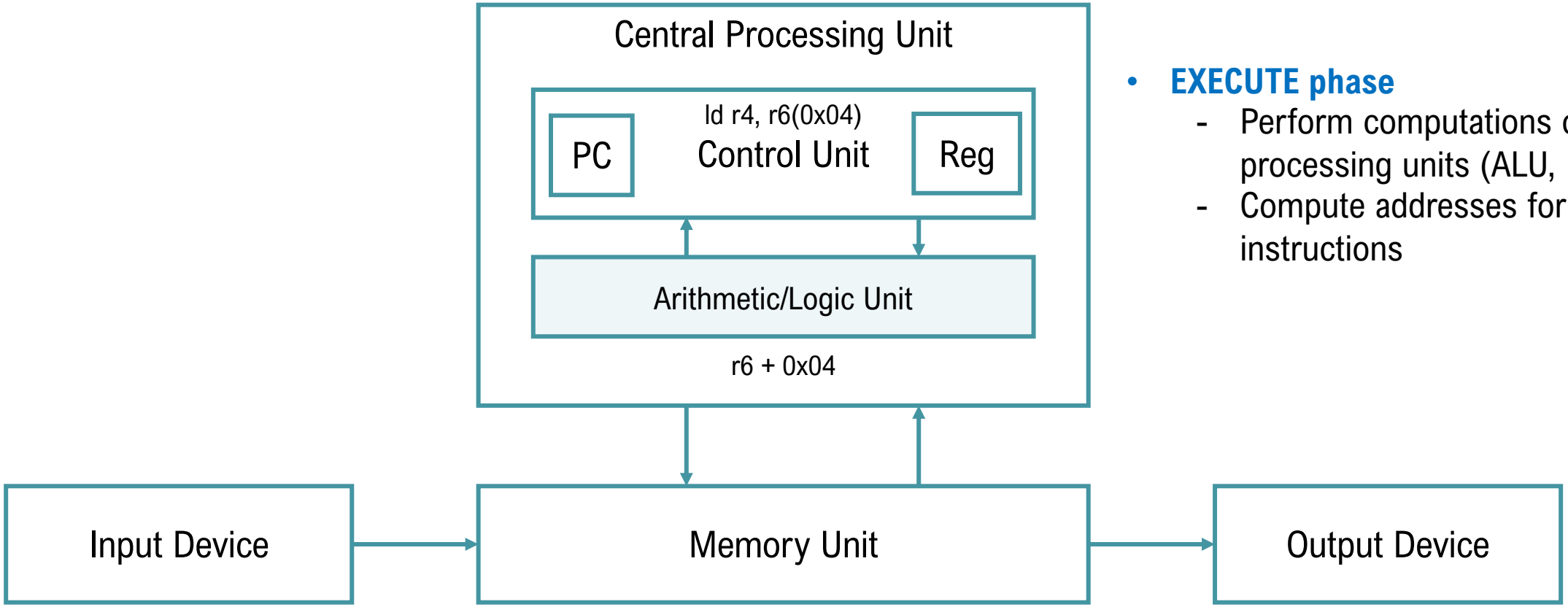
Read r2 and r3
Compute (r2 + r3)
Store the result to r1

Single-Instruction, Single-Data (SISD)



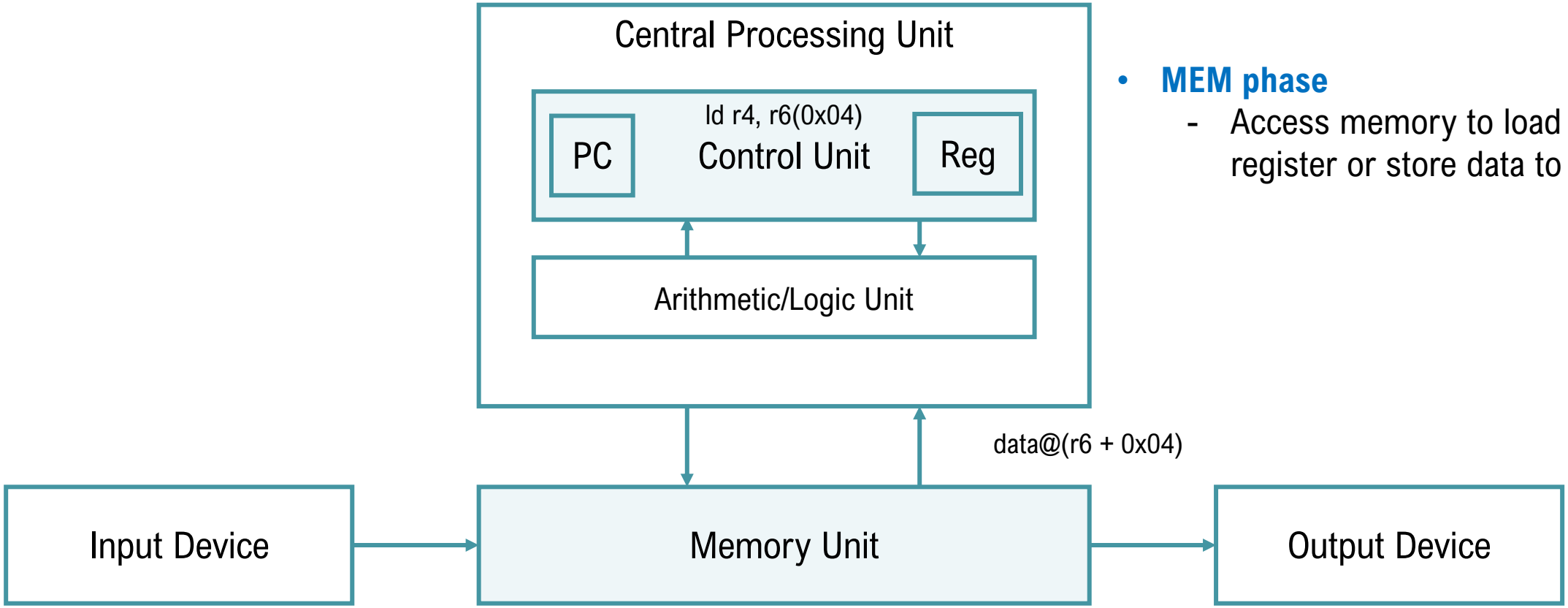
- **EXECUTE phase**
  - Perform computations on processing units (ALU, FPU)

Single-Instruction, Single-Data (SISD)



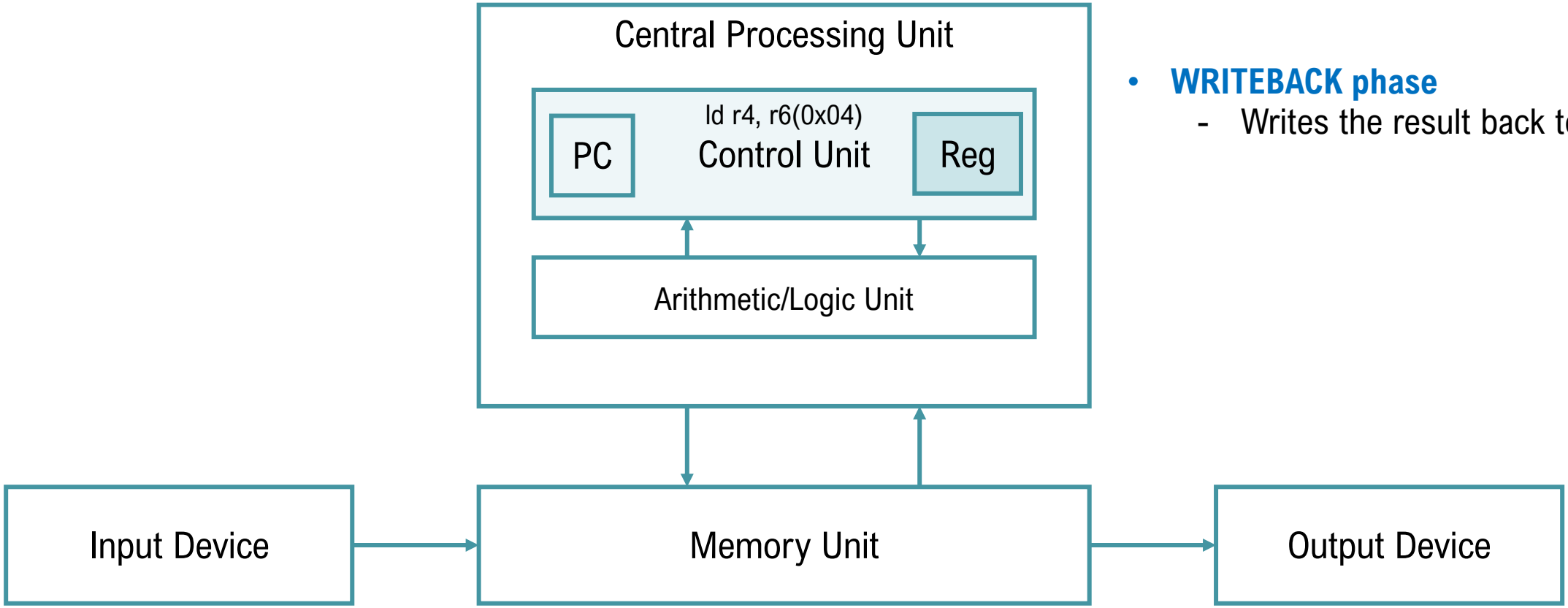
- **EXECUTE phase**
  - Perform computations on processing units (ALU, FPU)
  - Compute addresses for load/store instructions

Single-Instruction, Single-Data (SISD)



- **MEM phase**
  - Access memory to load data to a register or store data to memory

Single-Instruction, Single-Data (SISD)

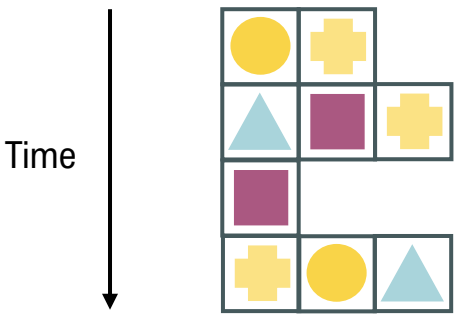


- **WRITEBACK phase**
  - Writes the result back to a register

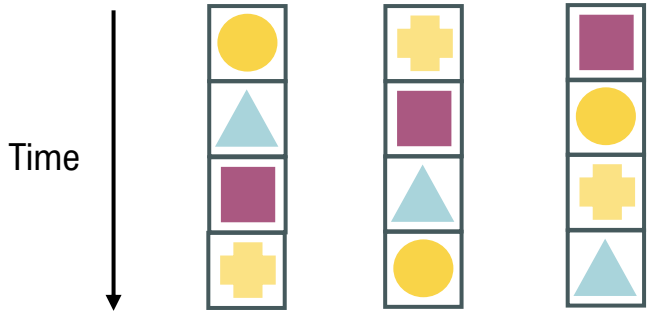
Types of Parallelism

- Instruction-level Parallelism (ILP)
  - Between individual, independent instructions
  - Hardware can look for **implicit** ILP within an instruction-window size
  - Compilers can re-organize the instructions so that those that fall within the window are more independent
- Thread-level Parallelism (TLP)
  - Compilers and programmers **explicitly** divide the program into multiple threads of control (each executing a set of instructions)
- Data-level Parallelism (DLP)
  - Constrained form of TLP where multiple threads of the same program operate on different data

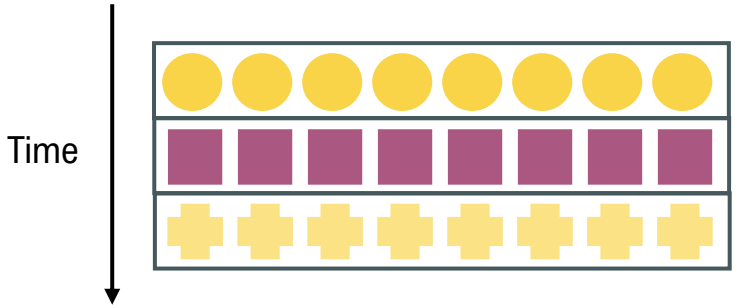
Types of Parallelism



Instruction-level parallelism (ILP)

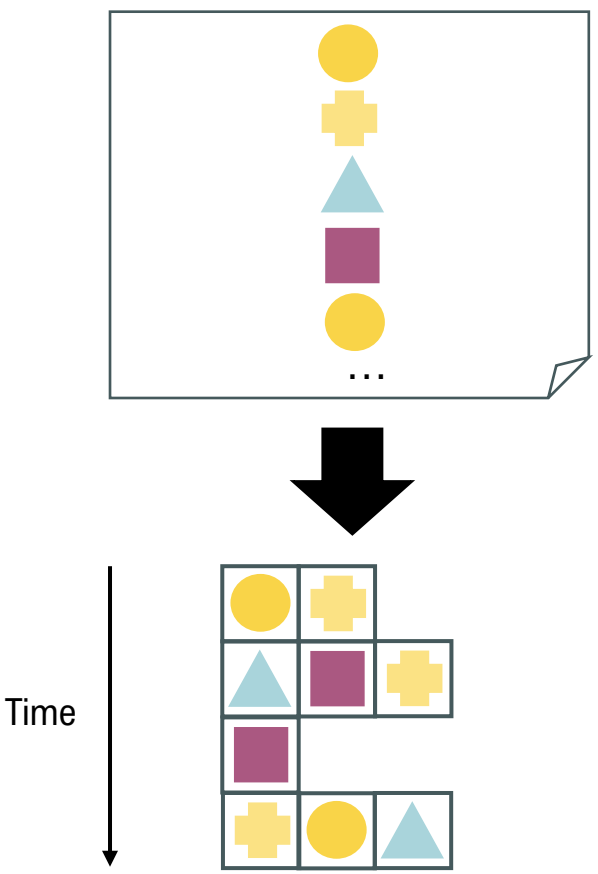


Thread-level parallelism (TLP)

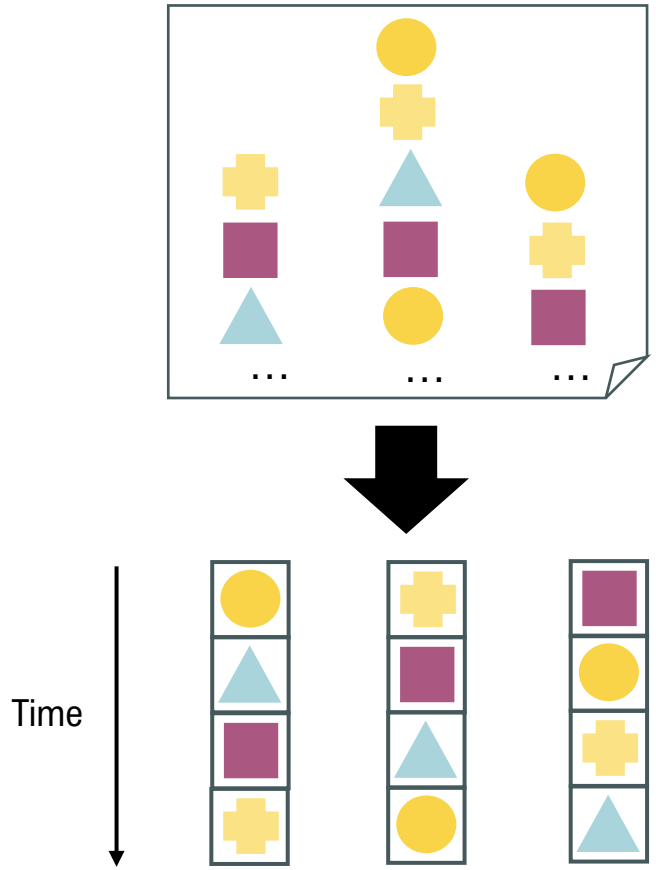


Data-level parallelism (DLP)

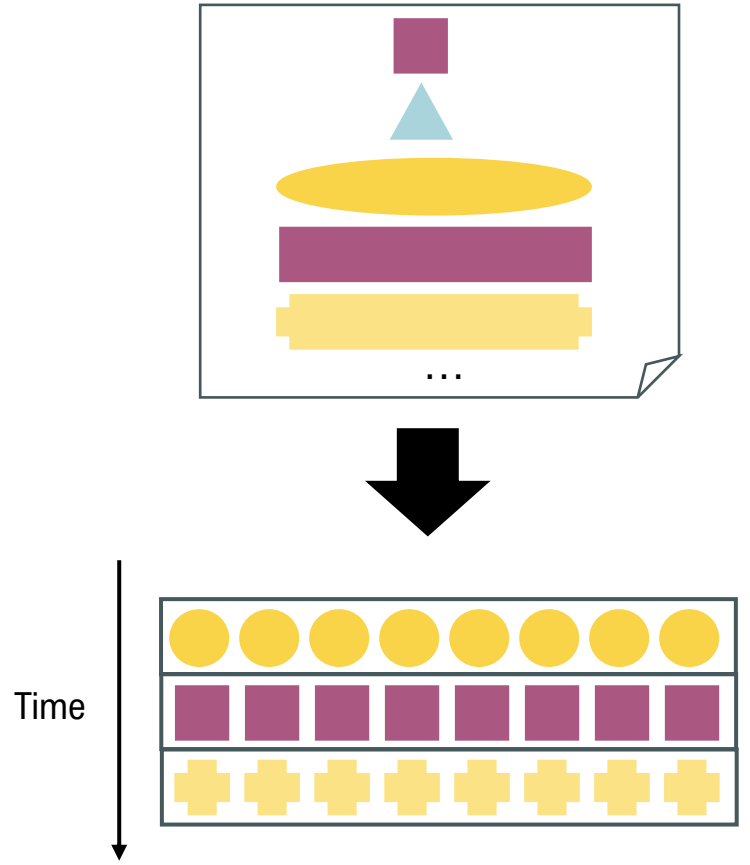
Types of Parallelism



Instruction-level parallelism (ILP)



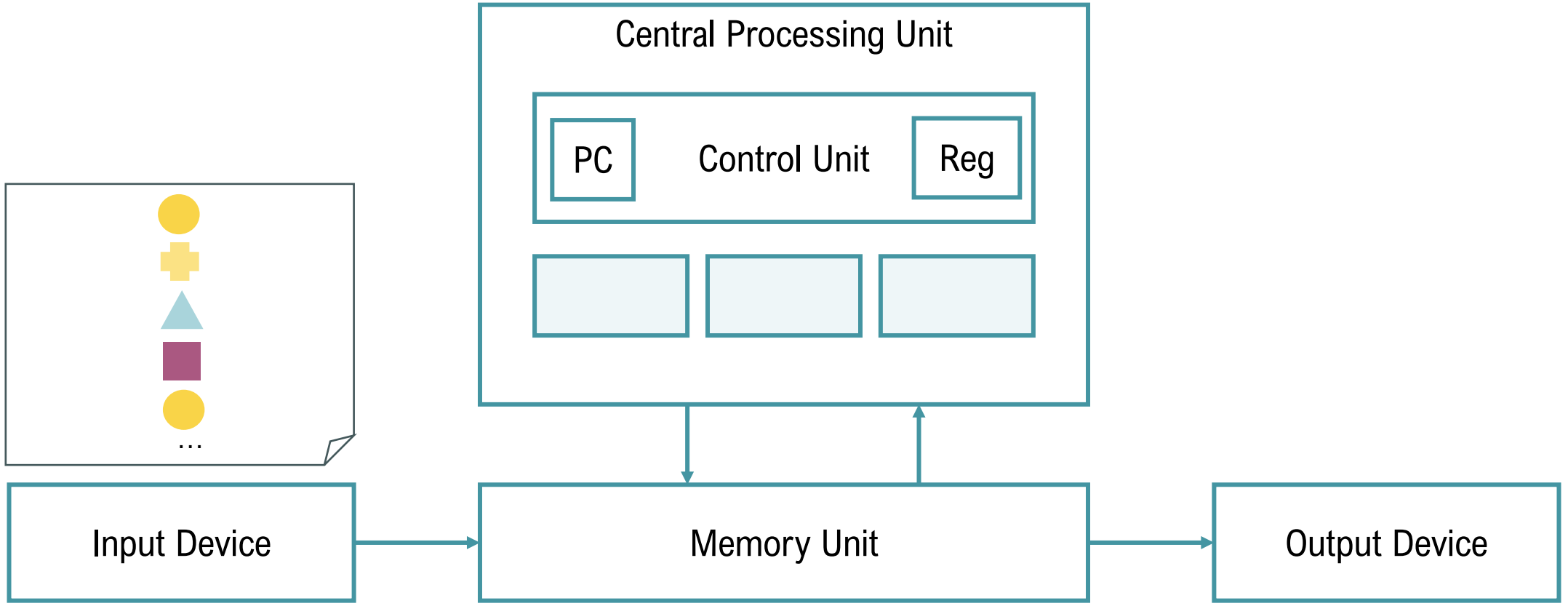
Thread-level parallelism (TLP)



Data-level parallelism (DLP)

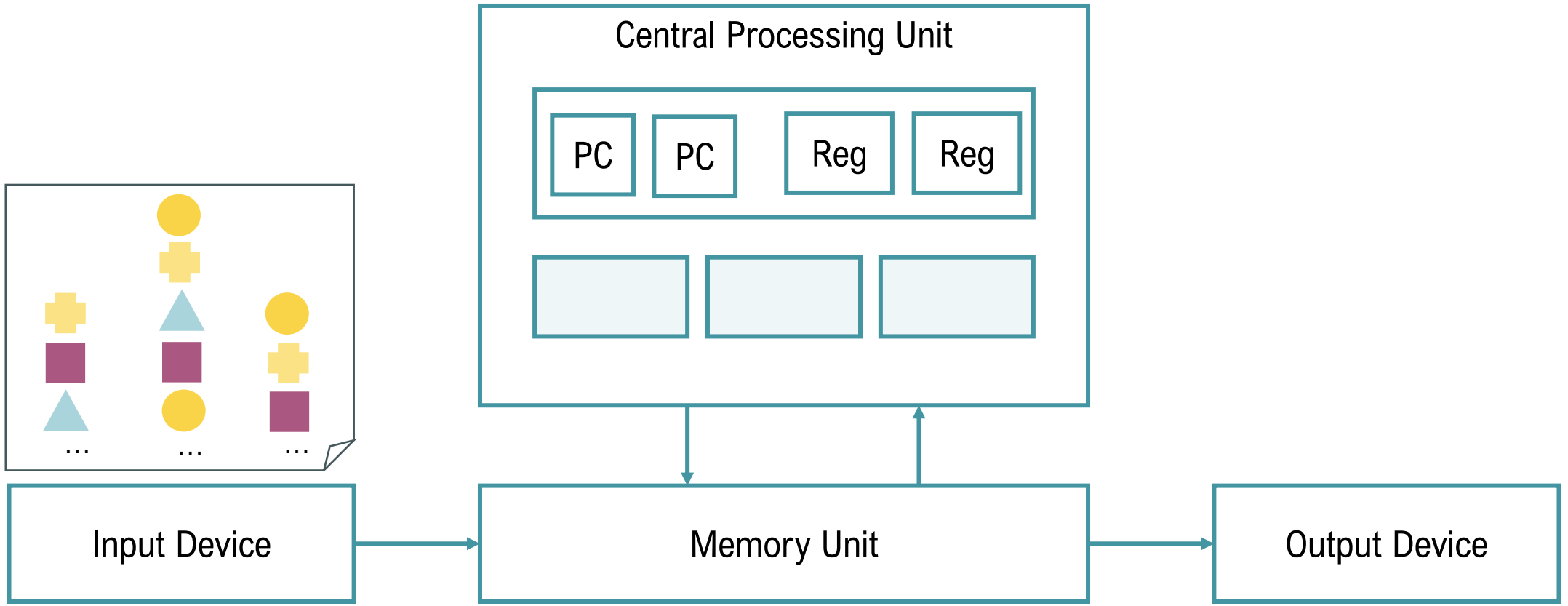
How to Exploit Parallelism in Hardware

- Instruction-level Parallelism (ILP):
  - Superscalar: Issue multiple instructions from a single thread. Limited by instruction dependencies



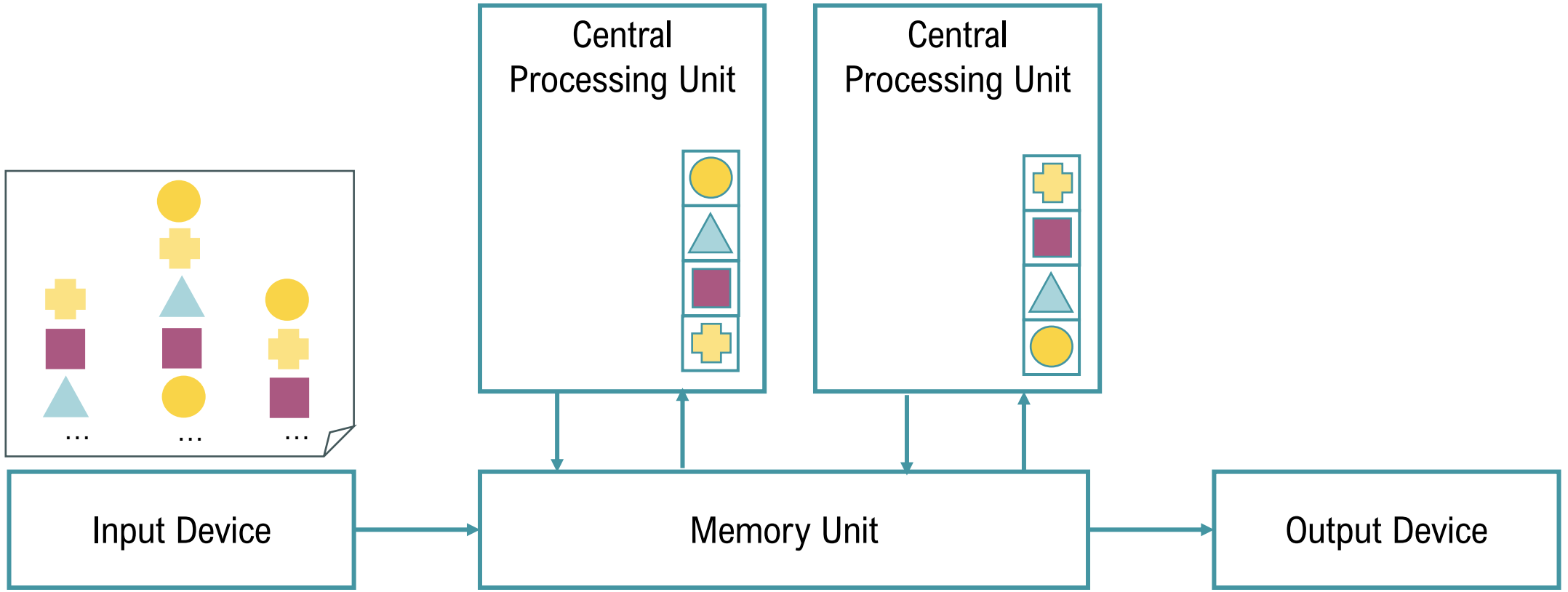
How to Exploit Parallelism in Hardware

- Thread-level Parallelism (TLP):
  - Fine-grained Multithreading: Fast context switching between threads. In any cycle, issue from one thread
  - Simultaneous Multithreading (SMT): Issue instructions from multiple threads in the same cycle



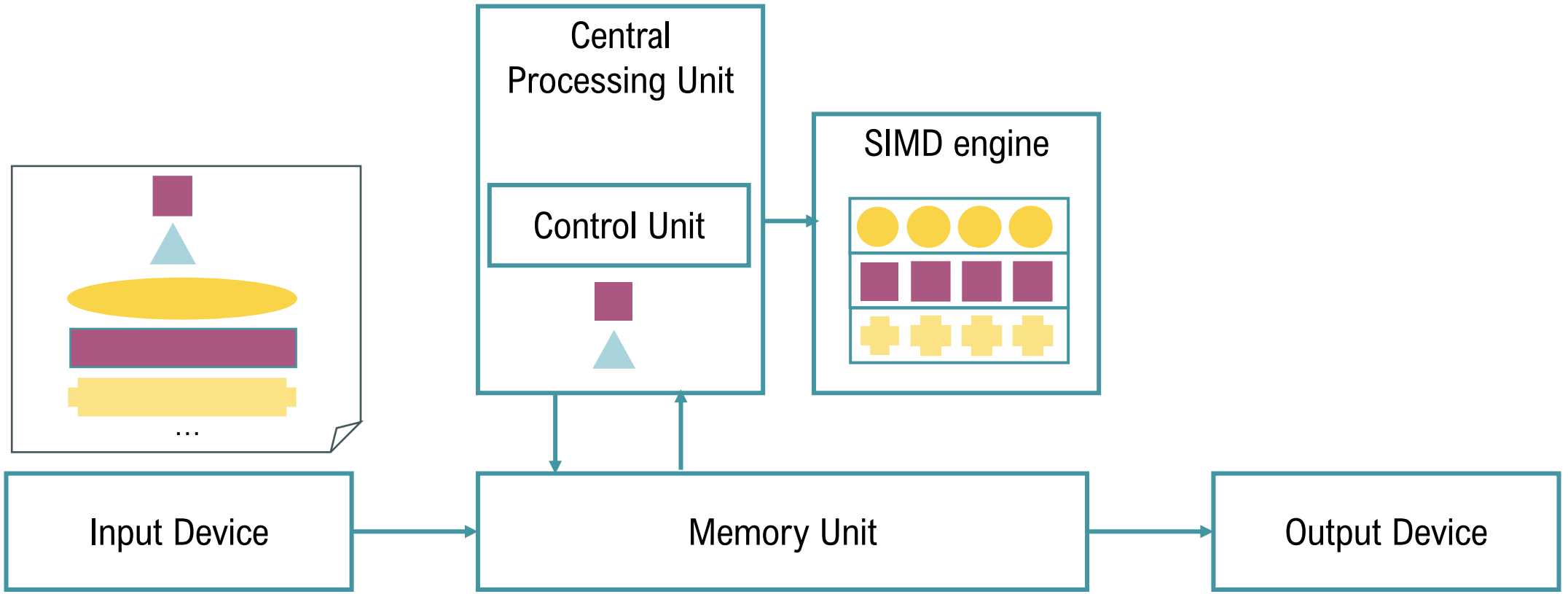
How to Exploit Parallelism in Hardware

- Thread-level Parallelism (TLP):
  - Chip Multi Processors (CMP): Multiple cores on a single chip. Each thread running on a different core



How to Exploit Parallelism in Hardware

- Data-level Parallelism (DLP):
  - Vector processors: Issues a single instruction that operats on multiple pieces of data



How to Exploit Parallelism in Hardware

- Instruction-level Parallelism (ILP):
  - Superscalar: Issue multiple instructions from a single thread. Limited by instruction dependencies
- Thread-level Parallelism (TLP):
  - Fine-grained Multithreading: Fast context switching between threads. In any cycle, issue from one thread
  - Simultaneous Multithreading (SMT): Issue instructions from multiple threads in the same cycle
  - Chip Multi Processors (CMP): Multiple cores on a single chip. Each thread running on a different core
- Data-level Parallelism (DLP):
  - Vector processors: Issues a single instruction that operats on multiple pieces of data

Microprocessor Architecture Trends

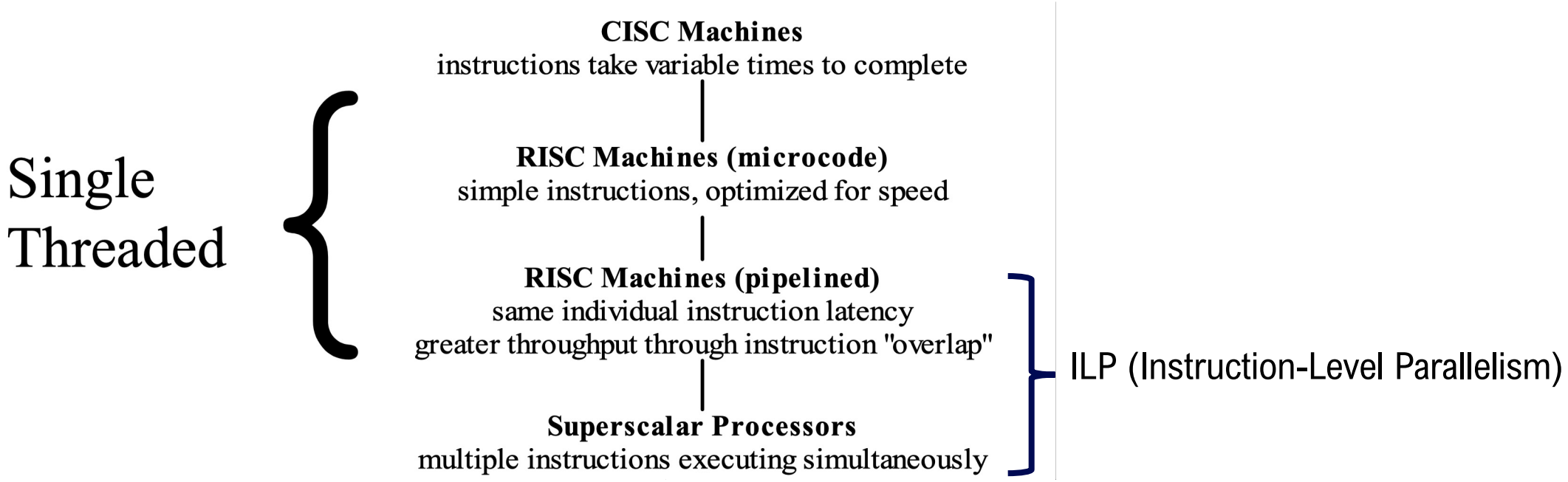
Single Threaded

CISC Machines
instructions take variable times to complete

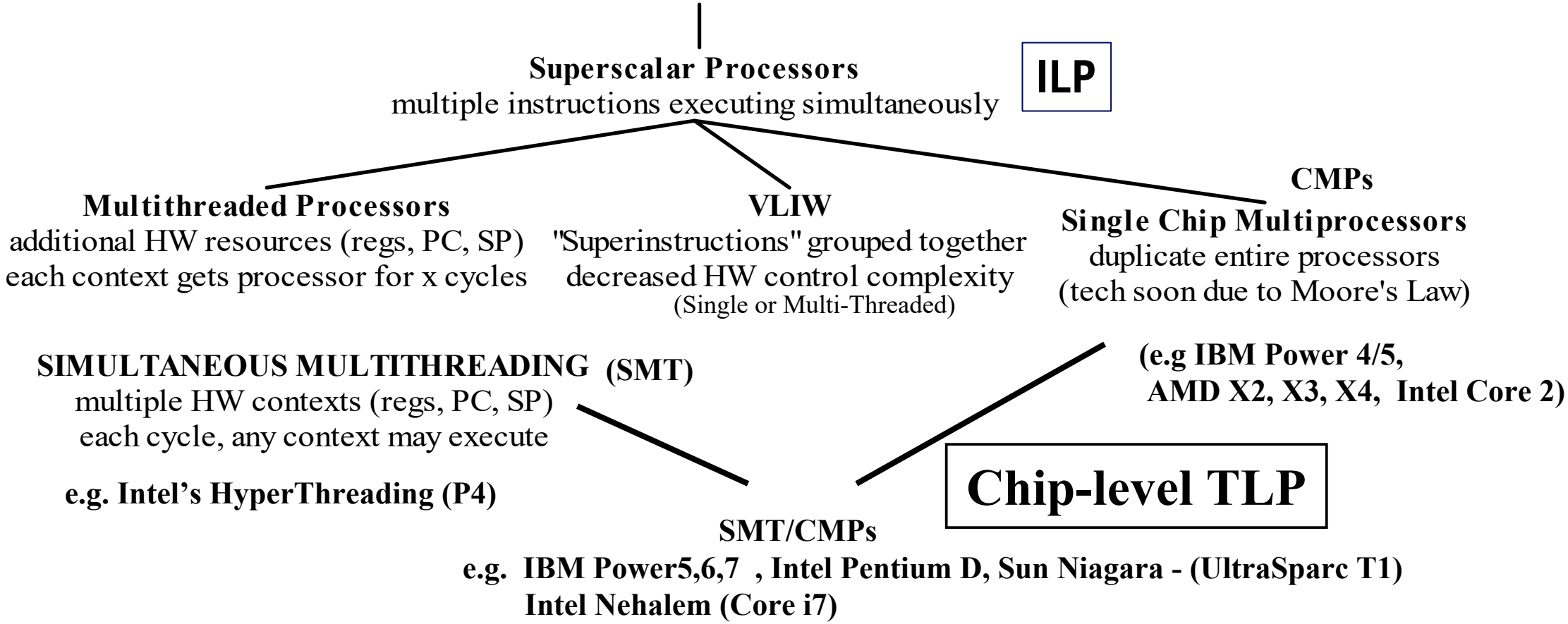
RISC Machines (microcode)
simple instructions, optimized for speed

RISC Machines (pipelined)
same individual instruction latency
greater throughput through instruction "overlap"

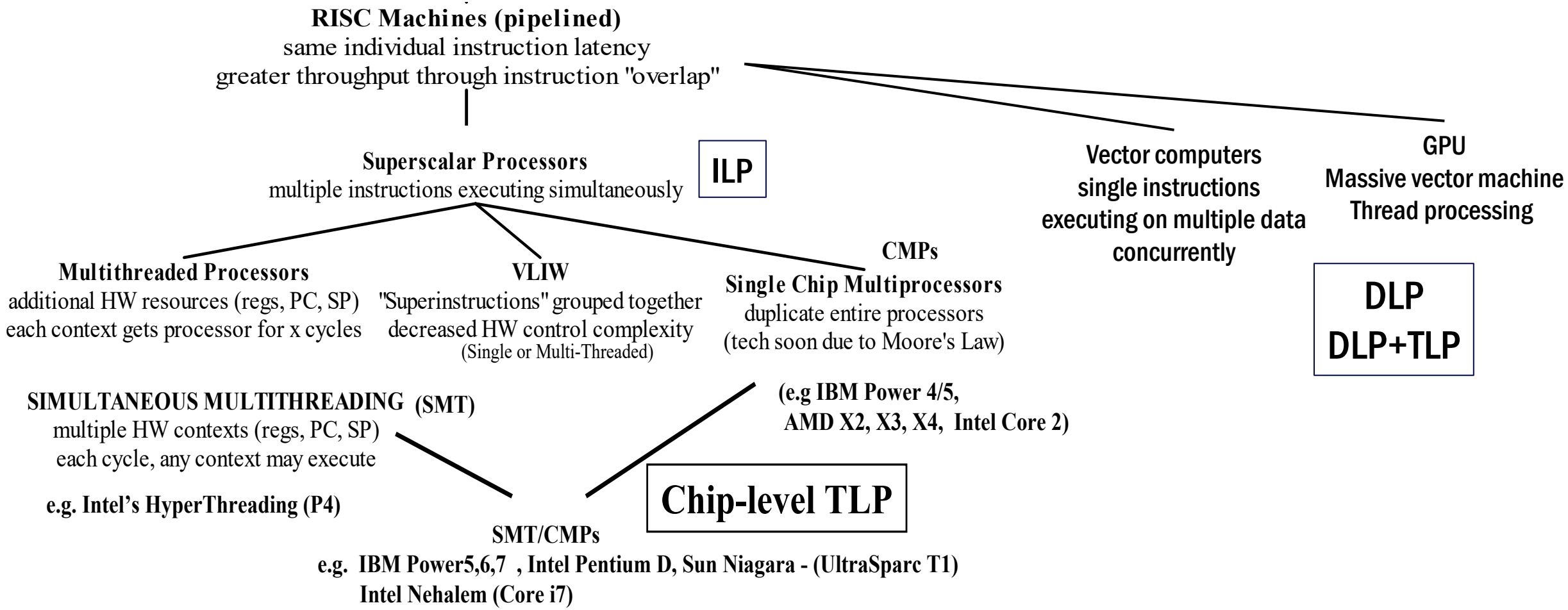
Microprocessor Architecture Trends



Microprocessor Architecture Trends



Microprocessor Architecture Trends



Model-based Classification

- Based on communication model
  - Shared-memory
  - Message-passing

- Based on execution model
  - Control-flow
  - Dataflow

Shared-Memory Architectures

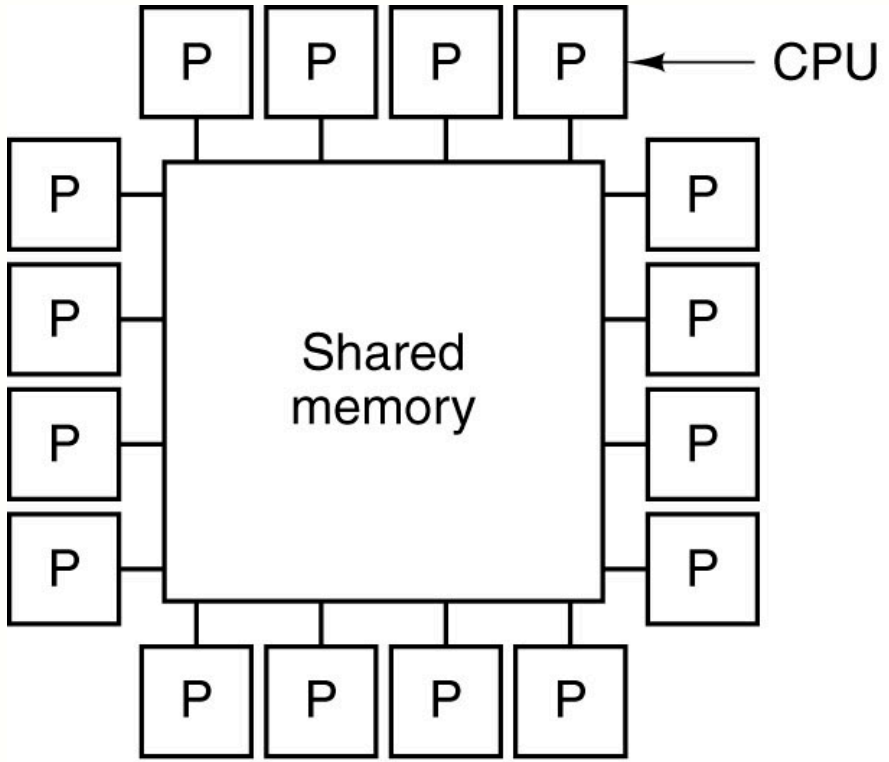
◎ Key feature: all processors in the system can directly access all memory locations in the system, thus providing a convenient and cheap mechanism for multiple processors to share data

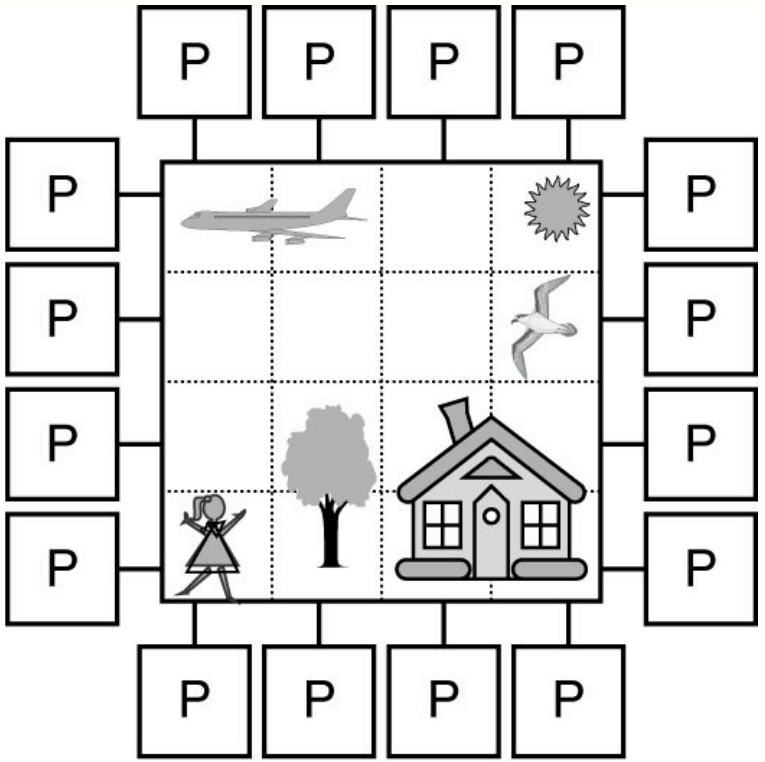
- Convenient: (1) location transparency, (2) abstractions supported is the same as that of uniprocessors
- Cheap: as compared to other models (more later)

◎ Memory can be centrally placed or distributed

Abstraction: Shared Memory Multiprocessors

- An architectural model simple and easy to use for programming
  - An address space is shared across cores
- Scalability issue in physical implementations





Shared-Memory Architectures

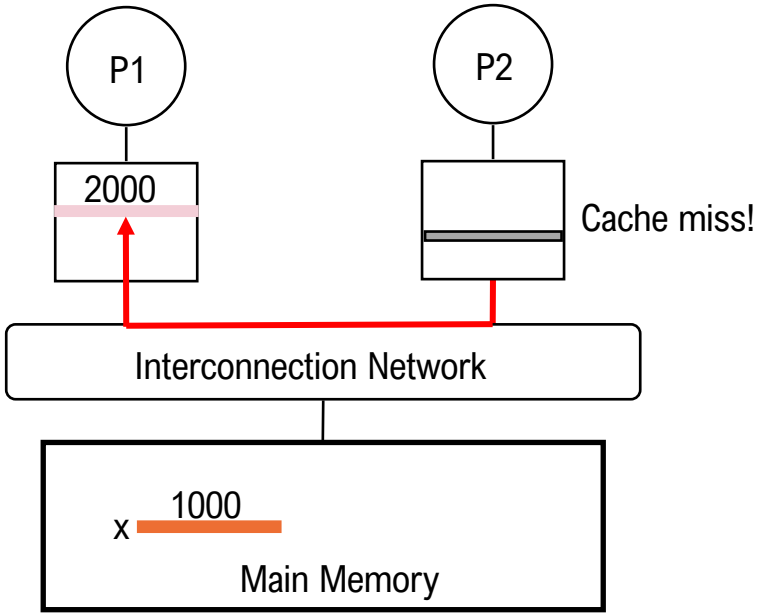
- Load and store instructions used to communicate data between processes
  - No OS involvement
  - Low software overhead
  - Prone to parallel programming bugs (e.g., data races)

- Special synchronization primitives to protect accesses to shared data
  - Hardware atomic instructions (e.g., fetch and op, compare-and-swap) serializes concurrent accesses to the same address
  - Synchronization primitives (e.g., locks, condition variables, barriers) are implemented using these instructions

- How to handle cached copies across cores and non-uniform access latencies?

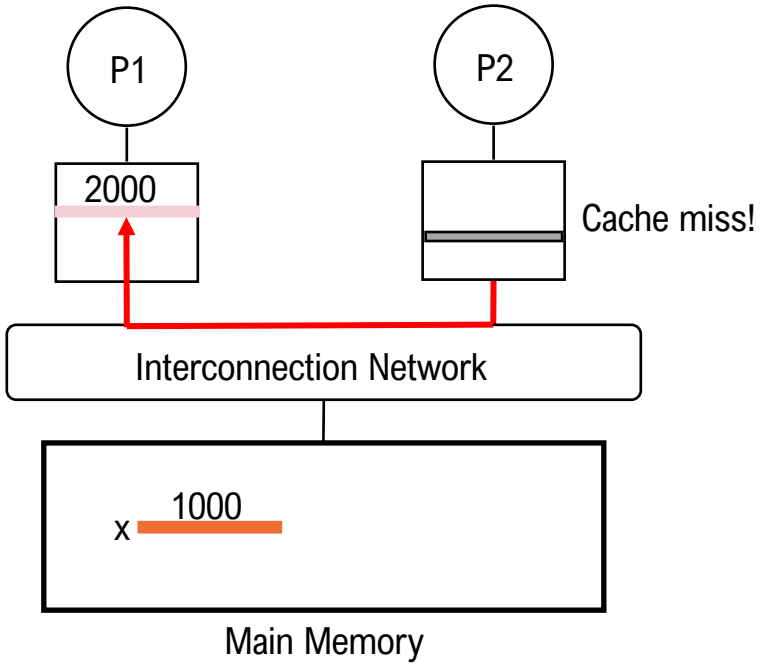
Cache Coherence

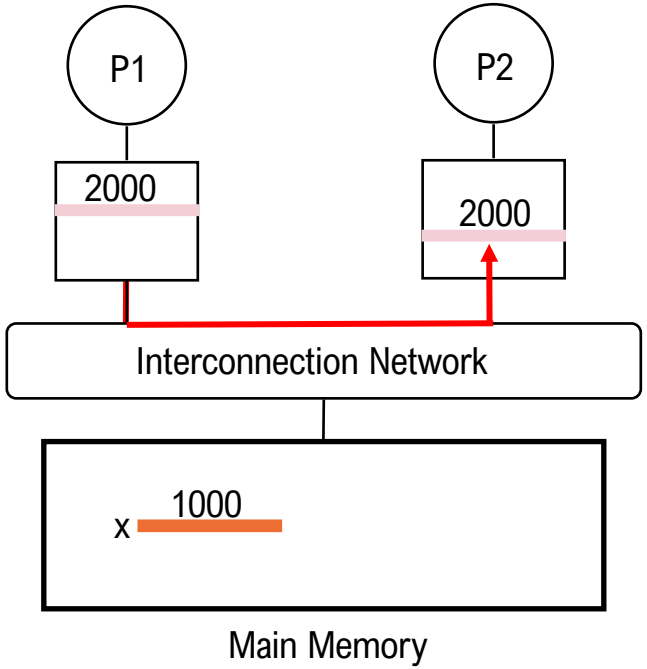
- Hardware cache coherence
  - Cores can retain local copies in their caches; what is are they shared across cores?
  - A cores/cache broadcasts its write/update to a memory location to all other cores, so that they can retrieve up-to-date values (coherent view of the shared space)
  - Another cache that has the location either updates or invalidates its local copy



Cache Coherence

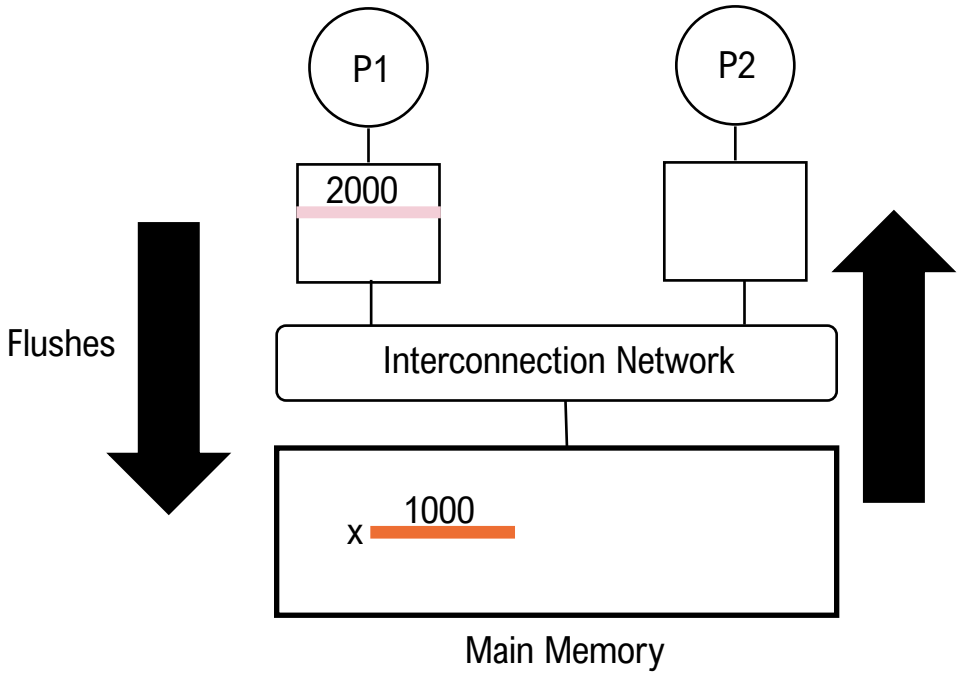
- Hardware cache coherence
  - Cores can retain local copies in their caches; what is are they shared across cores?
  - A cores/cache broadcasts its write/update to a memory location to all other cores, so that they can retrieve up-to-date values (coherent view of the shared space)
  - Another cache that has the location either updates or invalidates its local copy

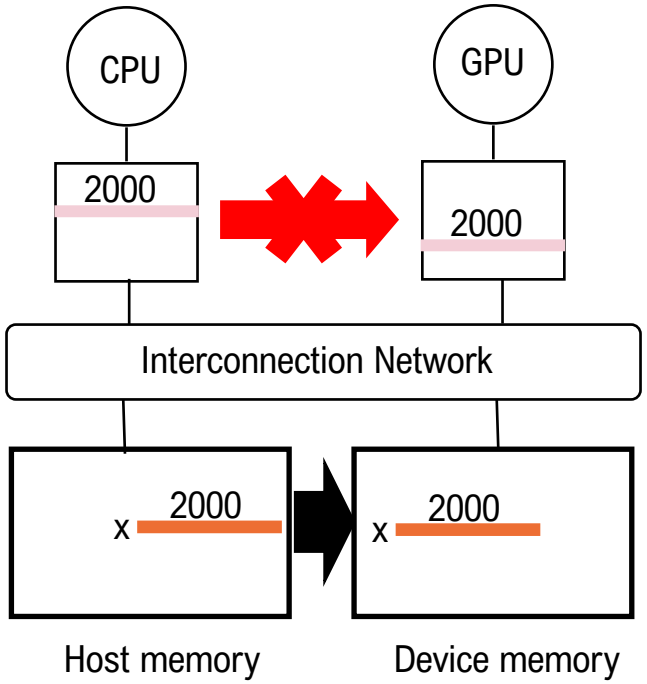




Cache Coherence

- Non cache coherent systems
  - Data sharing and communication occur at deterministic and known boundaries
  - Or coherence managed at the software level (e.g., OS pages)
  - There is no hardware support for propagating updated values





Software handoff (copy or migrate)

SMP vs. NUMA

- Symmetric multi-processor (SMP)
  - Uniform memory access time: cost of accessing an uncached memory address is the same for all processors
    - E.g., multiprocessors with centralized memory, chip multiprocessors

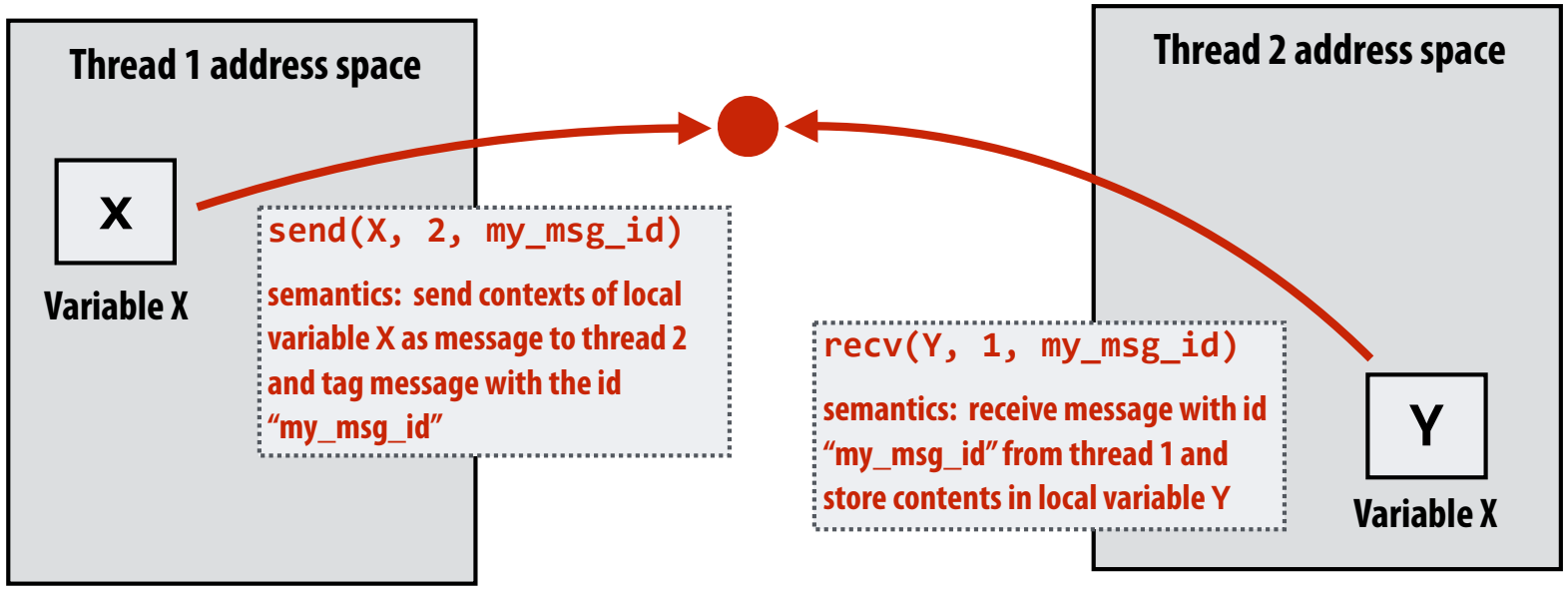
- Non-uniform memory access (NUMA)
  - All processors can access any memory location, but the cost of memory access (latency and/or bandwidth) is different for different processors
    - Addresses scalability issue
    - E.g., modern dual-socket configuration

Message-Passing Architectures

- Processors can directly access only local memory, and all communication and synchronization happens via messages
- Sending a message often incurs overheads
  - Building a header, copying data into network buffers, sending data, receiving data into buffers, copying data from kernel to user process address space
  - Many of these steps may require OS intervention
- Synchronization using messages typically based on various handshake protocols
- Advantage: easily scalable

Message-Passing Architectures

- Each process has a private address space
- Processes exchange data explicitly by sending and receiving messages
- Point-to-point communication transfers data and establishes ordering between the participating processes
- Collective operations coordinate groups of processes



(Communication operations shown in red)

Message-Passing Architectures

◎ Popular software library is MPI (message passing interface)

◎ Hardware need not implement system-wide loads and stores to execute message passing programs (need only be able to communicate messages)

- Can connect commodity systems together to form large parallel machine
- Message passing is a programming model for clusters

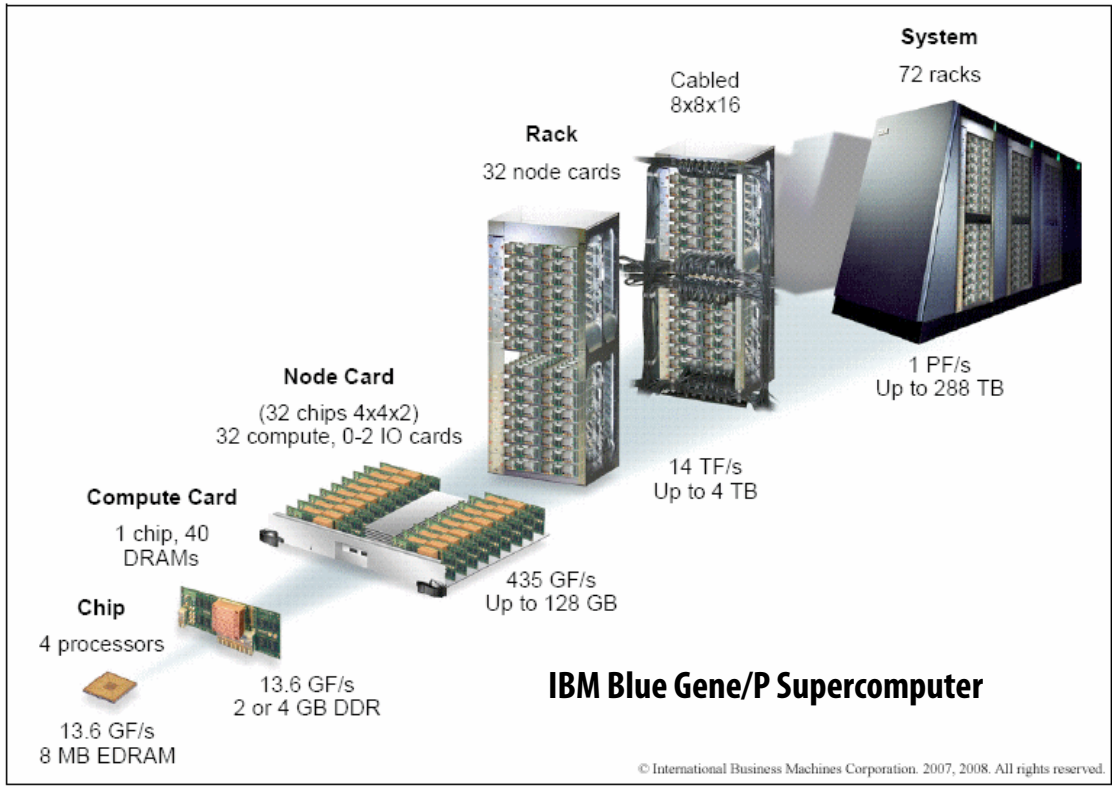


Image credit: IBM

https://research.ibm.com/publications/overview-of-the-ibm-blue-genep-project

Control-Flow vs. Dataflow

- Control-flow architectures
  - Instructions are fetched according to a program counter
  - Parallelism is extracted from or explicitly expressed within instruction streams
  - E.g., conventional CPUs and GPUs (=Von Neumann architectures)

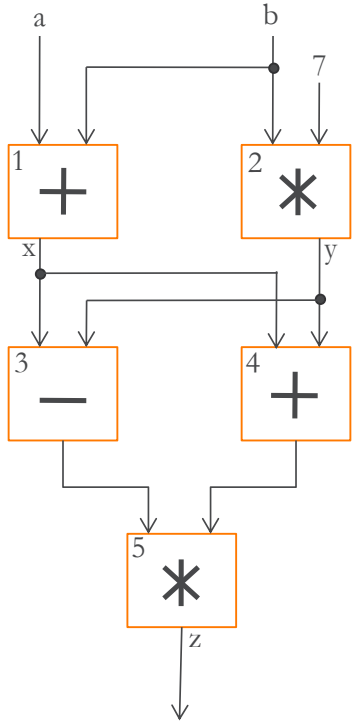
- Data-flow architectures
  - Computation is represented as a graph
  - Nodes represent operations, while edges represent data or dependencies
  - Independent nodes can execute concurrently when their inputs become available
  - E.g., Classic token-based dataflow machines, streaming FPGA/CGRAs, spatial accelerators

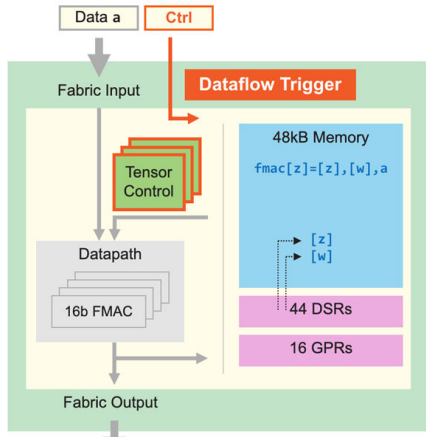
Dataflow Architectures

- In the dataflow model, instructions are activated by the availability of data operands for instructions
- Parallelism is naturally exposed, not hidden from hardware
- Many variations exist
  - Instruction-level dataflow architectures
    - No commercial products
    - Efficient handling of complex data structures like arrays
    - Complexity of the matching store
    - Problems due to excess of parallelism
  - Function-level (Coarse-grained) data-flow execution
    - Spatial architectures
    - FPGA/CGRA and ASIC-based accelerators

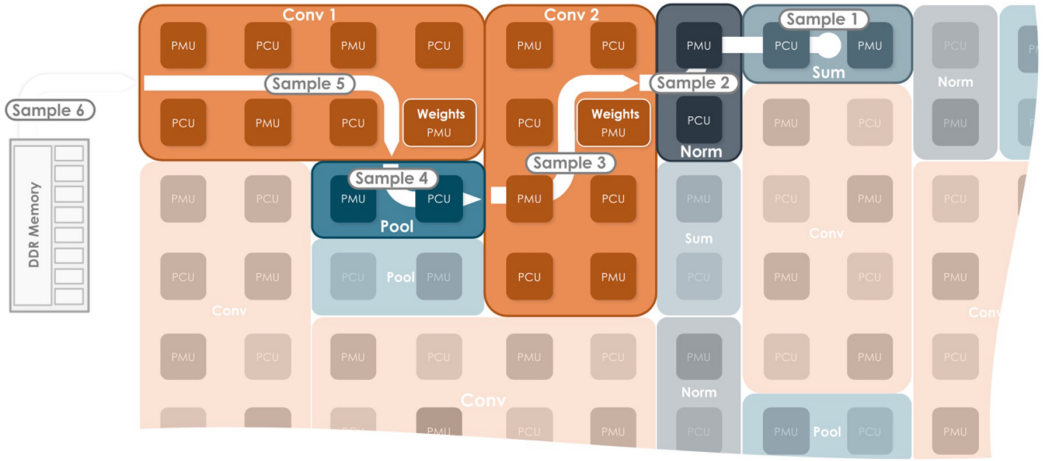
Dataflow Architectures

```
x=a+b;
y=b*7;
z=(x-y)*(x+y);
```





Cerebras

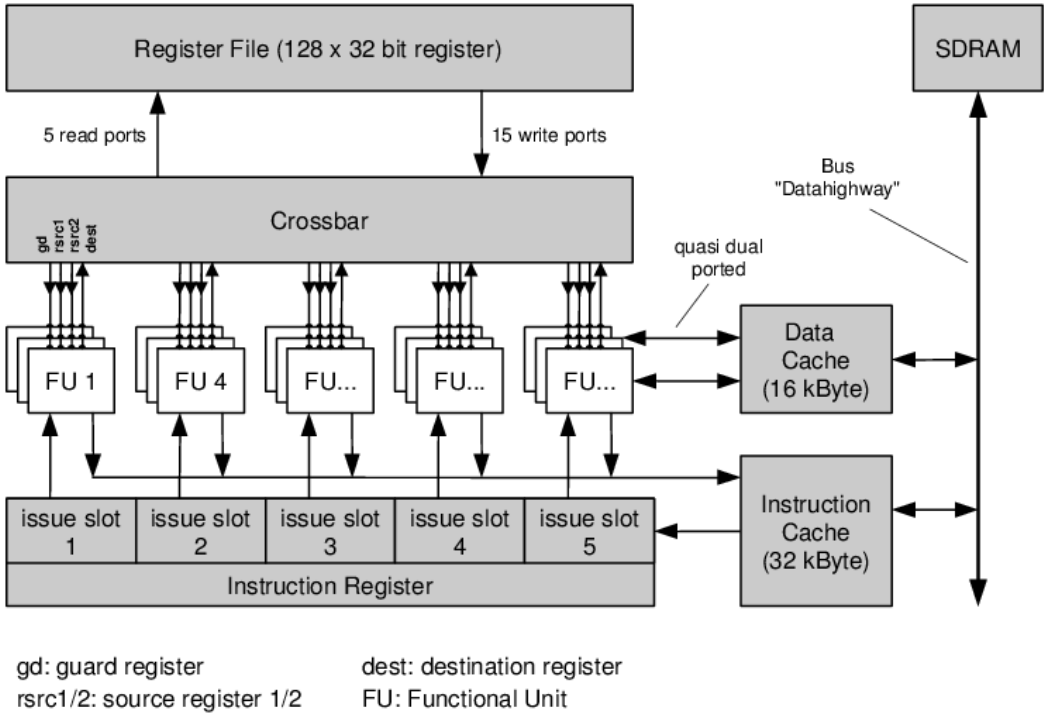


Sambanova

https://csg.csail.mit.edu/Users/arvind/ISCAfinal.pdf

Other Useful Classifications

- Homogeneous vs. heterogeneous
  - Multi-core CPU vs. CPU-GPU, CPU-NPU, big-little cores
- Static vs. dynamic scheduling
  - Static: compiler determines mapping and execution order
    - E.g., VLIW (Very Long Instruction Word) architectures
  - Dynamic: hardware or runtime makes decisions based on dependencies and resource availability



Programming Models vs. Machine Types

- The correspondence between programming models and machine types is fuzzy
  - Common to implement message passing abstractions on machines that implement a shared address space in hardware
    - Sending messages = copying memory from message library buffers
    - Receiving messages = copying data from messages library buffers
  - Can implement shared address space abstractions on machines that do not support it in HW (via less efficient SW solutions)
    - Mark all pages with shared variables as invalid
    - Page-fault handler issues appropriate network requests

Modern Practice: Mixed Parallelism

◎ Out-of-order CPU cores dynamically schedule ready instructions, approximating dataflow execution to exploit instruction-level parallelism

- Control-flow and dataflow

◎ Within a multicore node, threads communicate through a shared address space. Across nodes, processes communicate through message passing

- Shared memory and message processing

◎ Modern GPUs can execute multiple kernels concurrently. Each kernel exploits massive data parallelism across many threads

- TLP + DLP

Summary

◎ Programming models provide a way to think about the organization of parallel programs

- They provide abstractions that admit many possible implementations

◎ Restrictions imposed by these abstractions are designed to reflect parallelization and communication costs

- Shared-memory architectures: hardware supports any processor accessing any address
- Message-passing architectures: may have hardware to accelerate message send/receive/buffering
- It is desirable to keep “abstraction distance” low so that programs have predictable performance, but want it high enough for code flexibility/portability

Next Class

Introduction or Parallel Computing 2